Project 3: Finding a Person



By

Muhammad Ovais Yusuf

Lalitha Bhavanand

Torni Abrija

Samuel Korda

Robotics S25 (COMP 570-001)

Master of Science in Computer Science (MS CS)

Bridgewater, Massachusetts

(2025)

Introduction

In this project, we used TurtleBot4 and ROS2 to create an autonomous robotic system that uses facial recognition to detect and move towards a specific human target. This project's goal is to recreate a realistic human-robot interaction situation in which the robot must find a known person in a static environment, move in their direction, and signal successful recognition. This application may find application in autonomous service robotics, elder care, and search and rescue operations. Our objective was to create a unified system that successfully operates in real-world scenarios by combining computer vision, autonomous navigation, and hazard detection.

Hardware

The TurtleBot4 is a compact and extensible research robot designed for ROS2-based development. Key features used in this project include:

OAK-D Camera

The OpenCV AI Kit Depth (OAK-D) camera is a powerful module that combines stereo depth sensing and high-resolution RGB imaging. Although the RGB stream was mostly used in this project for real-time face recognition, the onboard depth capabilities offer a great deal of room for improvement in the future, especially in the areas of obstacle avoidance, gesture tracking, and spatial comprehension.

iRobot Create 3 Base

The Create 3 base, which serves as the robot's mobile platform, has a built-in IMU, wheel encoders, and sensors for cliff, bump, and wheel drop detection. It also provides reliable differential-drive

mobility. With the help of these safety features, the robot was able to safely explore its tabletop surroundings while reacting quickly to any physical obstacles or possible fall hazards.

Onboard computation and ROS2 support

The whole sensory, decision-making, and actuation pipeline was able to function without relying on an external workstation because to the robot's onboard computer, which was completely integrated with ROS 2. This standalone system demonstrated a completely autonomous robotic system with visual perception and safe navigation by processing camera data, performing facial recognition inference, and producing real-time movement commands.

Software

Our software architecture was built in Python and deployed using ROS2 Foxy. The system design was modular, separating vision processing, motion control, and safety mechanisms.

The main programming for this project was done in Python, which allowed for rapid development and integration with ROS2 packages. For image processing, we utilized the OpenCV library to handle frame resizing, format conversion, and pre-processing before face detection. This combination of Python and OpenCV proved effective for building a modular, real-time vision pipeline compatible with ROS2 nodes.

Below is an overview of the software components and their functionality:

Facial Recognition

We employed the face_recognition library, which uses deep learning-based models under the hood (ResNet-34) to generate 128-dimension facial embeddings. Incoming frames from the camera

were analyzed for facial features, and embeddings were compared using standard distance metrics. If a match was found, a bounding box was drawn, and the system confirmed the target visually.

Camera and Image Handling

The robot subscribed to the RGB image topic using sensor_msgs/Image. Images were converted using CvBridge to OpenCV format, resized, and passed into the face detection pipeline. The system managed to process frames approximately every 2 seconds, which was acceptable for a static environment.

Navigation System

Motion was controlled by publishing Twist messages to the /cmd_vel topic. The robot used the relative position of the detected face in the frame to adjust its heading, using a simple proportional controller. Linear velocity remained low to prevent overshooting the target.

Hazard Avoidance

Through the /hazard_detection topic, the robot continuously monitored for cliff sensors. If a drop was detected, it stopped forward movement and rotated to find a safe direction. This reactive behavior provided an essential layer of safety during autonomous operation.

LED Feedback

To confirm detection of the target person, the robot published a green blinking signal to the /cmd_lightring topic. This helped visually verify that the system had identified the correct individual. Overall, the software effectively combined visual recognition with real-time motion control, forming a functional prototype for person-finding behavior in constrained environments.

Pitfalls

One major challenge was obstacle detection beyond cliffs. While cliff sensors worked well, ground-level objects like cables and small items often escaped detection. These low-lying or dynamic obstacles led to occasional bumps and misalignment with the detected face.

We also encountered edge cases after overriding the cliff sensor's default halt behavior. Although this allowed the robot to rotate and find safer directions, if it remained near the edge for too long without new visual input, the system reverted to halt mode due to internal failsafe. This highlighted the importance of frequent sensor updates and watchdog mechanisms.

Another issue was the loss of face tracking due to environmental drift. Minor collisions often shifted the robot's orientation just enough to lose sight of the target face. In such cases, the robot abruptly switched back to search mode, interrupting the task. The absence of recovery logic, such as re-centering or tracking recent face positions, made this more disruptive.

At times, the system also suffered from camera frame lag or drop, especially under high load or lighting changes. This delay affected detection speed and led to hesitation or erratic behavior during navigation, as the robot relied heavily on real-time visual input.

Main Discussion

Experience and Demonstration Reflection

During the demonstration, the robot was placed randomly on the table. The known person (the target) and several others, including the professor, were intentionally positioned closer to test the system's ability to correctly identify the target.

- The robot initially performed a sweep of the environment using its camera.

- It ignored non-matching faces and only responded when it spotted the correct individual.
- Once the known face was detected, it calculated the face position, adjusted its heading, increased its speed and moved directly towards the target.
- As it approached, it used the LiDAR sensor to estimate safe distance and stopped smoothly when it was close enough.
- When it encountered a simulated cliff, it successfully rotated in place to avoid falling, then resumed scanning and navigation.

The demonstration ran smoothly under controlled, static conditions. The robot behaved as expected, providing real-time visual feedback and adapting to face recognition and terrain changes.

Lessons Learned

This project reinforced the importance of sensor fusion in robotics. Relying solely on vision or proximity sensors would have resulted in either unsafe or unintelligent behavior. By integrating camera-based facial recognition with LiDAR and hazard sensors, the robot was able to safely and intelligently navigate the environment.

We also learned that overriding built-in behaviors must be done with caution. While modifying the cliff sensor's default halt behavior allowed for smoother demonstration performance, it introduced new risks. In certain cases, the robot could enter a state of continuous rotation near an edge and fail to recover, highlighting the need for robust timeout handling and recovery strategies.

Another valuable insight was the need for recovery planning. When the robot lost sight of a partially visible face due to slight bumps or turns, it lacked the intelligence to continue pursuit or

reorient toward the last known position. Incorporating mechanisms like partial face memory or re-scanning protocols would have significantly improved reliability.

Interestingly, we found that simplicity often outperforms complexity in short-range static environments. Instead of fine-tuned angular corrections and dynamic path adjustments, simple straight-line motion with careful stopping distances proved to be more effective and consistent.

Real-time visual feedback was also invaluable during testing and debugging. Terminal logs and visual cues such as light ring feedback helped us understand what the robot was perceiving and how it was deciding its actions, making it easier to diagnose issues quickly.

Finally, we discovered that using LiDAR to stop at a safe distance from the target was more reliable than estimating proximity based on the apparent size of the detected face. This approach reduced false detections and improved repeatability across trials.

Conclusion & Evaluation

This project allowed us to explore the practical implementation of autonomous robotics using ROS2, with a specific focus on real-time vision-based target detection. The TurtleBot4 platform proved to be a dependable and accessible system for prototyping, especially for newcomers to robotic development. By integrating face recognition, autonomous navigation, and hazard detection, we were able to simulate a realistic human-robot interaction task that has potential applications in elder care, personal assistance, and search and rescue operations.

Throughout the development process, we deepened our understanding of the challenges and trade-offs involved in system integration, such as balancing simplicity with functionality, and ensuring real-time responsiveness under hardware constraints. We also learned that reliable behavior in demonstration conditions does not always translate to robustness in dynamic or unpredictable

environments. ROS2 enabled modular development, clearer debugging, and structured communication between perception and motion subsystems. Despite limitations in real-time performance due to onboard processing power, the robot successfully demonstrated autonomous behavior during the final demonstration and correctly identified the target individual as expected.

Ultimately, this project validated the feasibility of implementing person-finding logic on a compact autonomous robot and offered valuable insights into real-world robotics engineering. It also provided a solid foundation for future improvements, including behavior trees, better recovery logic, and integration of dynamic obstacle handling.